

STM32 F103C8T6

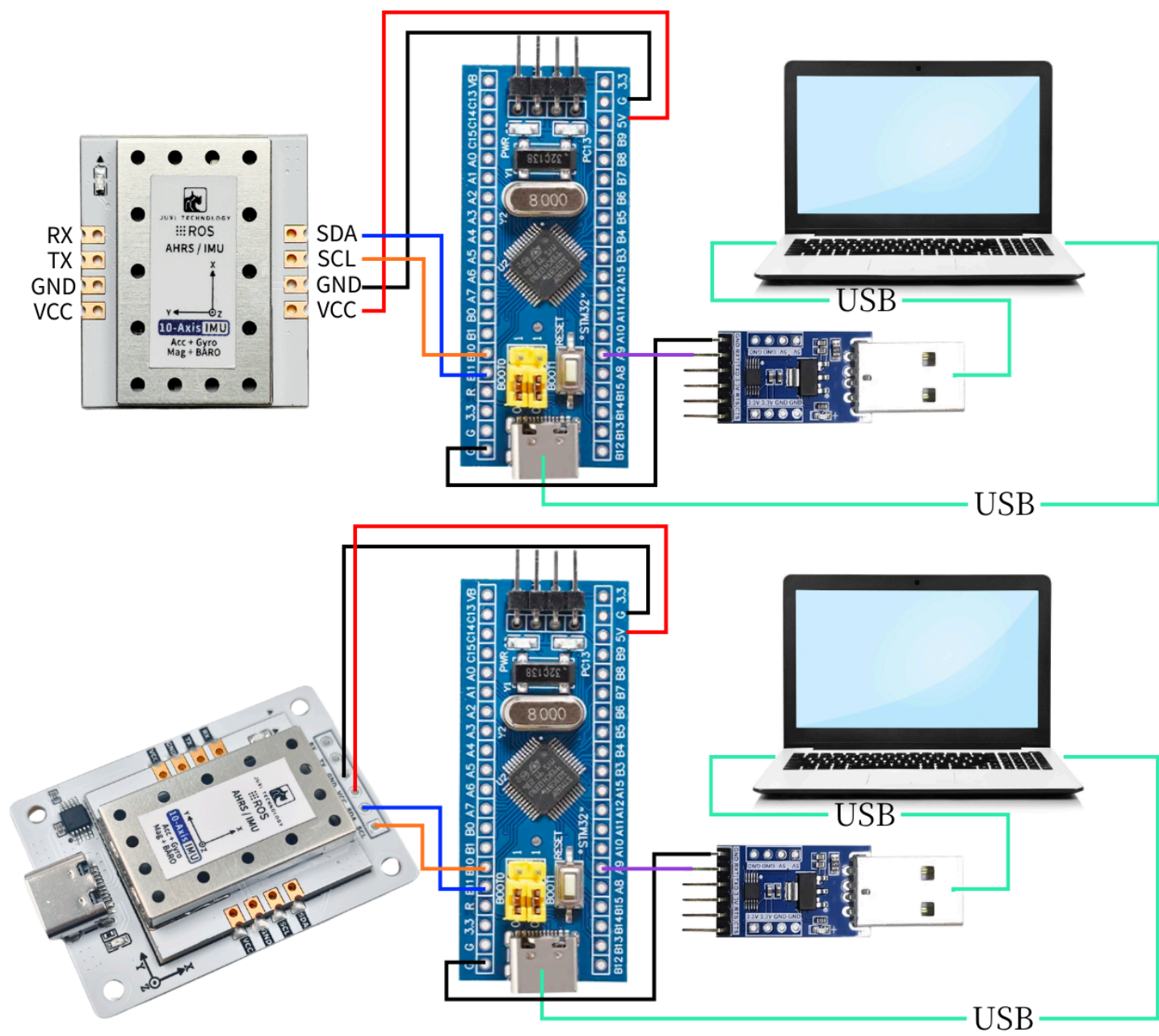
This routine uses an STM32F103C8T6 core board, a Windows computer, several DuPont wires, and an IMU attitude sensor.

 STM32.zip
13.63MB



Use Keil5 software to open USART.uvprojx, and burn the program into the STM32F103C8T6 core board

1. Connect the device




```

1 //解析环形缓冲中的数据，提取完整帧并更新缓存
2
3 //Process RX ring buffer, parse frames and update internal cache
4
5 void IMU_UART_Process(void)
6 {
7     enum {
8         RX_STATE_EXPECT_HEAD1 = 0,
9         RX_STATE_EXPECT_HEAD2,
10        RX_STATE_EXPECT_LENGTH,
11        RX_STATE_EXPECT_FUNCTION,
12        RX_STATE_COLLECT_DATA
13    };
14
15    static uint8_t rx_state = RX_STATE_EXPECT_HEAD1;
16    static uint8_t frame_length = 0;
17    static uint8_t frame_function = 0;
18    static uint8_t frame_buffer[64]; /* 数据区 + 校验 / data section +
checksum */
19    static uint16_t frame_index = 0;
20
21    uint8_t current_byte = 0;
22
23    // 处理环形缓冲区中的所有数据
24    while (_rxbuf_pop(&current_byte) == 0) {
25        switch (rx_state) {
26            case RX_STATE_EXPECT_HEAD1:
27                // 寻找帧头1
28                if (current_byte == FRAME_HEAD1) {
29                    rx_state = RX_STATE_EXPECT_HEAD2;
30                }
31                // 否则保持在当前状态
32                break;
33
34            case RX_STATE_EXPECT_HEAD2:
35                // 寻找帧头2
36                if (current_byte == FRAME_HEAD2) {
37                    rx_state = RX_STATE_EXPECT_LENGTH;
38                } else {
39                    // 帧头不匹配，重新开始寻找
40                    rx_state = RX_STATE_EXPECT_HEAD1;
41                }
42                break;
43
44            case RX_STATE_EXPECT_LENGTH:
45                // 保存帧长度
46                frame_length = current_byte;

```

```

47         rx_state = RX_STATE_EXPECT_FUNCTION;
48         break;
49
50     case RX_STATE_EXPECT_FUNCTION:
51         // 保存功能码
52         frame_function = current_byte;
53         frame_index = 0;
54         rx_state = RX_STATE_COLLECT_DATA;
55         break;
56
57     case RX_STATE_COLLECT_DATA: {
58         // 计算数据长度 (帧长度 - 帧头2字节 - 长度1字节 - 功能码1字节)
59         uint16_t data_length = (frame_length >= 4) ? (uint16_t)
60         (frame_length - 4) : 0;
61
62         // 检查数据长度是否有效
63         if (data_length == 0 || data_length > sizeof(frame_buffer)) {
64             rx_state = RX_STATE_EXPECT_HEAD1;
65             break;
66         }
67
68         // 存储当前字节
69         frame_buffer[frame_index++] = current_byte;
70
71         // 检查是否收集完所有数据
72         if (frame_index >= data_length) {
73             // 计算校验和
74             uint8_t calculated_checksum = (uint8_t)(FRAME_HEAD1 +
75             FRAME_HEAD2 + frame_length + frame_function);
76             for (uint16_t i = 0; i < data_length - 1; ++i) {
77                 calculated_checksum += frame_buffer[i];
78             }
79
80             // 验证校验和
81             uint8_t received_checksum = frame_buffer[data_length - 1];
82             if (calculated_checksum == received_checksum) {
83                 // 校验通过, 解析数据
84                 _parse_frame_data(frame_function, frame_buffer);
85             }
86
87             // 重置状态, 准备接收下一帧
88             rx_state = RX_STATE_EXPECT_HEAD1;
89         }
90     } break;
91
92     default:
93         // 未知状态, 重置

```

```

92         rx_state = RX_STATE_EXPECT_HEAD1;
93         break;
94     }
95 }
96 }
97
98
99 /* ----- 解析数据帧 / Parse one complete frame ----- */
100 static void _parse_frame_data(uint8_t frame_function, const uint8_t
101 *frame_data)
102 {
103     switch (frame_function) {
104         case IMU_FUNC_RAW_ACCEL: {
105             // 定义常量比例因子
106             const float ACCEL_RATIO = 16.0f / 32767.0f;
107             const float DEG2RAD = 3.14159265358979323846f / 180.0f;
108             const float GYRO_RATIO = (2000.0f / 32767.0f) * DEG2RAD;
109             const float MAG_RATIO = 800.0f / 32767.0f;
110
111             // 解析加速度数据
112             s_ax = to_int16(&frame_data[0]) * ACCEL_RATIO;
113             s_ay = to_int16(&frame_data[2]) * ACCEL_RATIO;
114             s_az = to_int16(&frame_data[4]) * ACCEL_RATIO;
115
116             // 解析陀螺仪数据
117             s_gx = to_int16(&frame_data[6]) * GYRO_RATIO;
118             s_gy = to_int16(&frame_data[8]) * GYRO_RATIO;
119             s_gz = to_int16(&frame_data[10]) * GYRO_RATIO;
120
121             // 解析磁力计数据
122             s_mx = to_int16(&frame_data[12]) * MAG_RATIO;
123             s_my = to_int16(&frame_data[14]) * MAG_RATIO;
124             s_mz = to_int16(&frame_data[16]) * MAG_RATIO;
125             break;
126         }
127         case IMU_FUNC_EULER:
128             s_roll = to_float(&frame_data[0]);
129             s_pitch = to_float(&frame_data[4]);
130             s_yaw = to_float(&frame_data[8]);
131             break;
132         case IMU_FUNC_QUAT:
133             s_q0 = to_float(&frame_data[0]);
134             s_q1 = to_float(&frame_data[4]);
135             s_q2 = to_float(&frame_data[8]);
136             s_q3 = to_float(&frame_data[12]);
137             break;
138         case IMU_FUNC_BARO:

```

```

138         s_height          = to_float(&frame_data[0]);
139         s_temperature      = to_float(&frame_data[4]);
140         s_pressure         = to_float(&frame_data[8]);
141         s_pressure_contrast = to_float(&frame_data[12]);
142         break;
143     case IMU_FUNC_VERSION:
144         s_version_high = frame_data[0];
145         s_version_mid  = frame_data[1];
146         s_version_low  = frame_data[2];
147         break;
148     case IMU_FUNC_RETURN_STATE:
149         s_last_rx_function = frame_data[0];
150         s_last_rx_state    = (int16_t)frame_data[1];
151         break;
152     default:
153         // 未知帧类型，可添加错误处理
154         break;
155 }
156 }

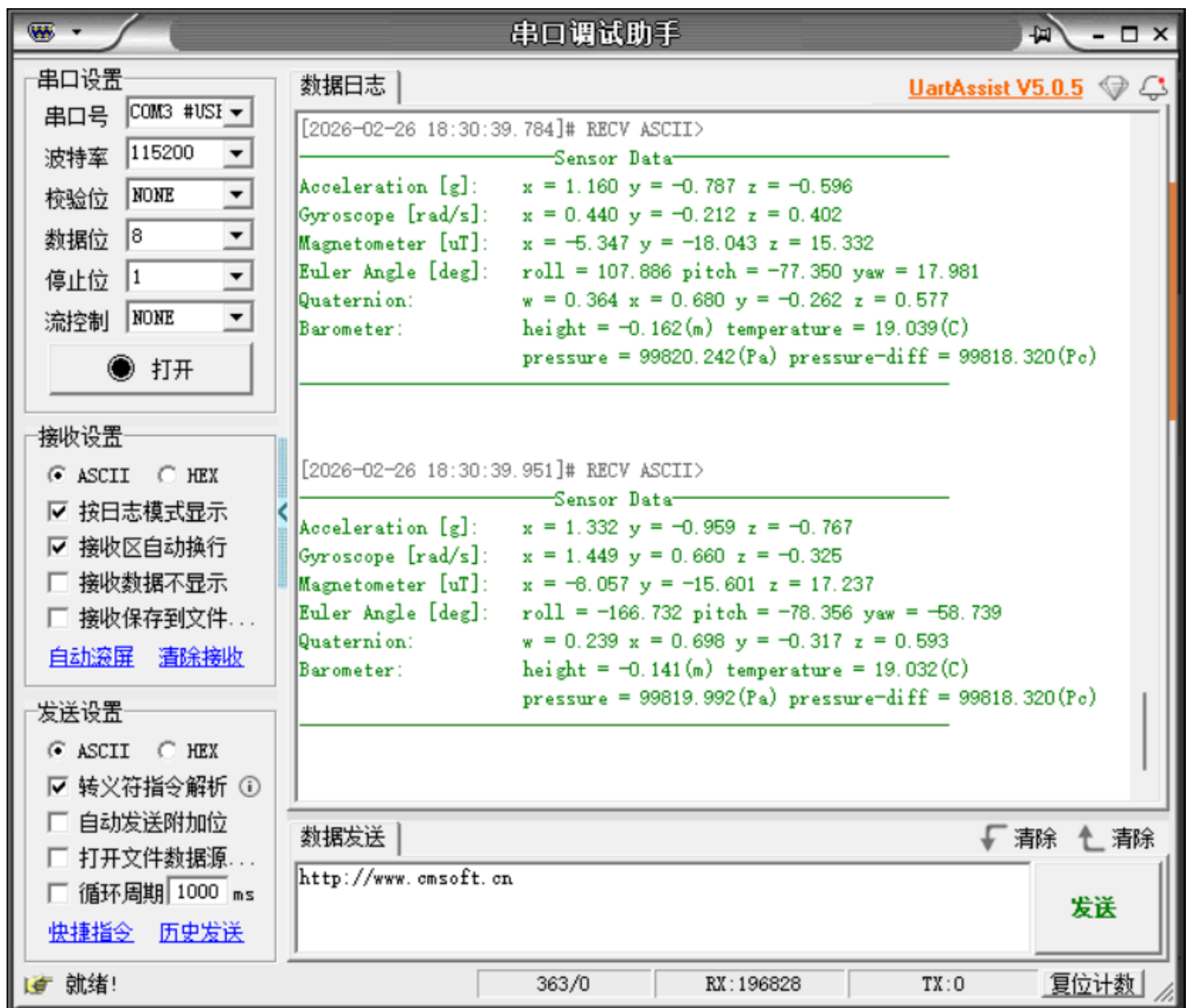
```

IMU_UART_Process(): Reads the data from the buffer and calls _parse_frame_data to parse the data that conforms to the Communication Protocol.

_parse_frame_data(): Parse data.

3. Read IMU data

After the program is downloaded into the STM32, open the serial port assistant (with configuration parameters as shown in the figure below), and you can see that the data of the IMU module is continuously printed. When we change the attitude of the IMU module, the data will change.



Note: The above is the data reading for a 10-axis IMU. The 6-axis has no Magnetometer and Barometer data, and the 9-axis has no Barometer data.